

TempoBridge: Language-Guided Tempo Control for Vision-Language-Action Policies

Yeonseo Lee¹, Hyosup Shin¹, Guebin Hwang¹, and Sungho Jo^{1†}

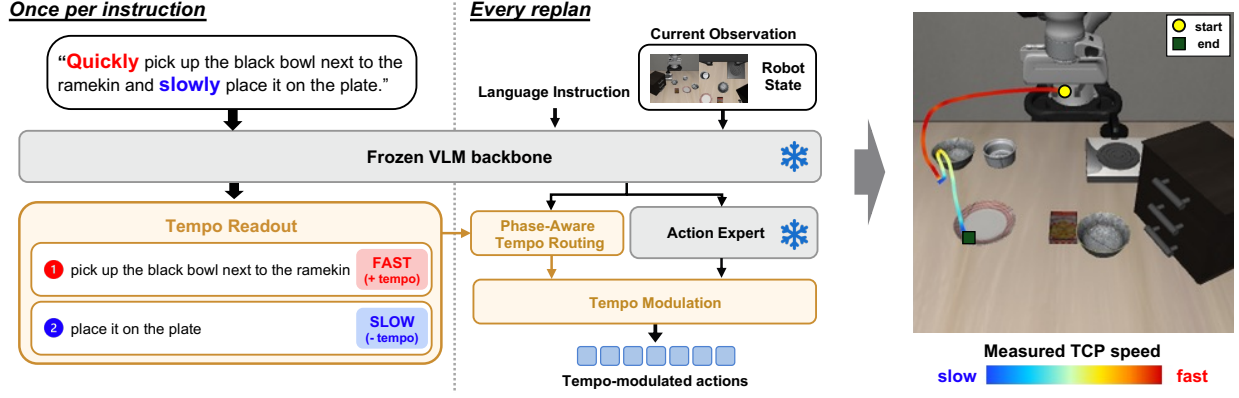


Fig. 1. Language-conditioned tempo control with TempoBridge. TempoBridge reads tempo cues from the internal representations of a frozen VLA policy and aligns them with task progress to adjust execution speed according to the instruction.

Abstract—Vision-Language-Action (VLA) models are effective at understanding *what* task to perform, but provide limited control over *how* it should be executed, such as moving quickly or slowly. We introduce TempoBridge, a lightweight framework that uses frozen VLA representations to modulate actions according to tempo cues in the instruction at each task phase, without additional tempo-conditioned robot demonstrations or tempo-specific base-policy fine-tuning. TempoBridge extracts tempo cues from contextual VLM representations, aligns them with task progress through a causal phase router, and modulates nominal motion commands during execution. Across LIBERO tasks, TempoBridge improves Tempo Success Rate from 52.6% to 89.7% under canonical tempo instructions while retaining high task success. It also preserves near-baseline performance when no tempo cue is present and generalizes to unseen tempo expressions without additional training. Experiments on a physical robot further demonstrate language-conditioned tempo modulation in real-world manipulation. Project page: <https://lysees.github.io/tempo-bridge-page/>

I. INTRODUCTION

Vision-Language-Action (VLA) models have recently emerged as a promising paradigm for general-purpose robot manipulation, enabling a single policy to execute diverse tasks from natural-language instructions [1], [2], [3]. Existing VLAs increasingly understand what to do, but provide limited control over how to execute it [4], [5]. In many practical settings, however, execution manner is an important part of the user’s intent [6], [7]. For example, a robot may be asked to move *quickly* for efficiency or *slowly* for deliberate execution. These preferences may also vary as a task progresses, as in “*quickly pick up the object, then slowly*

place it on the target.” In this work, we study this capability as *language-conditioned tempo control*.

A straightforward approach is to train a policy on trajectories associating tempo instructions with execution speeds. However, collecting demonstrations at different speeds is costly and difficult to scale across tasks, objects, and environments [8], [9], [10]. Temporal augmentation can reduce the cost of collecting additional demonstrations [4], but incorporating the augmented trajectories into the policy still requires computationally expensive fine-tuning [11].

Pretrained VLA representations may already encode semantic distinctions associated with manner expressions such as quickly and slowly, and prior work has shown that internal features related to such concepts can be identified and manipulated to influence VLA behavior [5]. This raises the possibility of using semantic tempo information available in the frozen representation as an interface for execution control, rather than learning tempo interpretation entirely from newly collected robot trajectories. The key challenge is to connect these semantic tempo cues to an explicit execution-time control mechanism while accounting for task progress. We therefore ask: *Can semantic tempo cues from a frozen VLA be connected to execution-time motion modulation, enabling language-conditioned tempo control without tempo-specific robot demonstrations or policy fine-tuning?*

To address this question, we introduce **TempoBridge**, a lightweight framework that reads semantic tempo information from a frozen VLA’s internal language representations and connects it to phase-aware motion modulation. TempoBridge takes a frozen $\pi_{0.5}$ policy [12] as its base policy and uses a *prototype-based tempo readout* to extract a compact tempo sequence from the instruction. When the requested tempo varies within a task, a *causal phase router* aligns this

¹Korea Advanced Institute of Science and Technology (KAIST), Daejeon, Korea.

[†]Corresponding author.

sequence with task progress using visual features reused from the policy and robot-state history. The selected tempo then modulates the nominal motion commands during closed-loop execution. The base VLA continues to generate the nominal task actions, while TempoBridge modulates their execution according to the requested tempo and estimated task phase.

This design keeps the pretrained policy fixed while separating tempo interpretation and phase estimation from nominal action generation. The tempo readout is constructed from text examples, whereas the phase router is trained on existing task demonstrations annotated with phase-transition intervals, without tempo labels. Consequently, TempoBridge requires neither tempo-conditioned robot demonstrations nor tempo-specific fine-tuning of the base policy. When no tempo cue is detected, the readout returns NORMAL and the nominal actions remain unchanged. Because the readout operates on contextual VLM representations, the same fixed readout can also be applied to previously unseen tempo expressions without retraining.

We evaluate TempoBridge on 40 manipulation tasks across the Spatial, Object, Goal, and Long suites of LIBERO [13]. TempoBridge raises Tempo Success Rate from 52.6% to 89.7% under canonical quickly/slowly instructions, while preserving near-baseline performance without tempo cues and improving tempo compliance for unseen expressions. Real-robot experiments further validate language-conditioned tempo modulation.

Our main contributions are:

- We formulate language-conditioned tempo control for frozen VLAs and investigate whether semantic tempo cues in their internal representations can serve as an interface for execution-time motion modulation, without additional tempo-conditioned robot demonstrations or tempo-specific base-policy fine-tuning.
- We introduce **TempoBridge**, a lightweight framework that combines prototype-based tempo readout, causal phase routing, and execution-time action modulation while keeping the entire base policy frozen.
- We evaluate physical tempo compliance, task competence under original instructions, and transfer to unseen tempo expressions across 40 LIBERO tasks, and further validate TempoBridge in real-world manipulation experiments with a physical robot.

II. RELATED WORK

A. Vision-Language-Action Models

Vision-Language-Action (VLA) models transfer semantic and world knowledge from pretrained vision-language models into robotic control. RT-2 [1] established this paradigm by co-finetuning web-scale VLMs with robot trajectories in a shared token space, and subsequent generalist policies scaled language-conditioned manipulation across heterogeneous embodiments [14], [2] or coupled pretrained representations with continuous action experts for dexterous, long-horizon execution [3], [12]. Across these paradigms, however, language specifies *what* should be achieved, while

how an action is executed remains implicit in the demonstration distribution. Temporal characteristics such as speed or caution, therefore, do not surface as explicit control variables, even though the corresponding linguistic concepts are plausibly already represented in the pretrained backbone.

B. Controlling Execution in Pretrained VLAs

Approaches to controlling *how* a policy executes differ mainly in what part of the system they modify, and therefore in what supervision they require.

One line modifies data and weights. Language motions as an intermediate action representation, counterfactual relating, and semantically aligned action tokenizers [7], [15], [16] all learn how a linguistic modifier maps to physical execution from robot trajectories. TempoVLA [4] conditions on an explicit speed scalar and re-times demonstrations by temporal augmentation, with a VLM scheduler dispatching per-chunk speeds at deployment. This still requires retraining the policy, and tempo arrives as a numeric input from an external module.

A second line intervenes on a frozen policy’s internal activations, differing in how the directions are identified [5], [17], [18] and how the intervention is regulated once found [19], [20]. Häon *et al.* [5] show that linguistic concepts such as *slow* survive VLA training in the backbone and remain causally linked to end-effector motion, though they identify features heuristically by token similarity and steer by overriding their activations with a selected coefficient rather than a physically interpretable execution variable. Buurmeijer *et al.* [17] make such intervention systematic by training linear observers for robot states and actions and computing the minimal representation-space edit that places an observed feature inside a desired interval. In both cases, the target is an externally specified number rather than read from language, and identifying the directions requires either manual inspection or labeled robot data. CTRL-STEER [19] instead regulates the magnitude of the intervention online with a feedback controller, although the target concept is still supplied externally and the controller is fitted per task.

In contrast, TempoBridge reads tempo cues from the language instruction through frozen VLA representations and connects them to phase-aware motion modulation, without tempo-conditioned demonstrations or tempo-specific VLA fine-tuning.

III. METHOD

A. Overview and Problem Formulation

We introduce **TempoBridge**, a lightweight framework that interprets language-specified tempo using contextual representations from a frozen vision-language-action (VLA) policy and connects it to phase-aware motion modulation. As illustrated in Fig. 2, TempoBridge uses a frozen $\pi_{0.5}$ policy [12] as its base policy, consisting of a PaliGemma [21] VLM backbone and an action expert. We use *tempo* to denote the qualitative execution pace associated with a task event. At each replanning step, the base policy produces a nominal

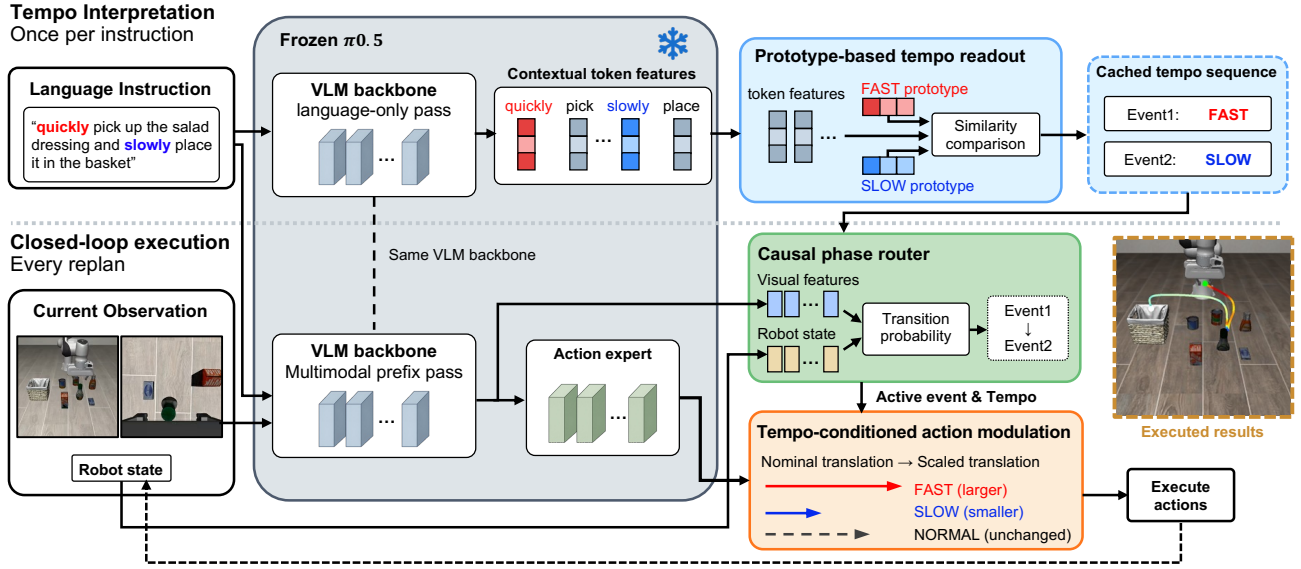


Fig. 2. Overview of TempoBridge. Using the frozen VLM backbone within $\pi_{0.5}$, a prototype-based tempo readout extracts an ordered tempo sequence from the instruction once and caches it for execution. At each replan, a causal phase router uses visual features reused from the policy forward pass and robot state to track task progress and select the active event. The corresponding tempo coefficient scales the translational components of the nominal action chunk.

action chunk from the current observation and language instruction, while all of its parameters remain fixed.

TempoBridge consists of three components: a prototype-based tempo readout, a causal phase router, and tempo-conditioned action modulation. The readout processes the instruction once and extracts a cached *tempo sequence* $z \in \mathcal{Z}$, where

$$\mathcal{Z} = \{[\text{NORMAL}], [\text{FAST}], [\text{SLOW}], [\text{FAST}, \text{SLOW}], [\text{SLOW}, \text{FAST}]\}.$$

A single-entry sequence specifies one tempo throughout execution. A two-entry sequence specifies the requested tempo order over two successive task events, for which we assume a single forward transition between events.

During closed-loop execution, the causal phase router tracks task progress and selects the active entry of the cached tempo sequence. The corresponding tempo is then used to modulate the nominal action chunk. For a single-entry sequence, the router is bypassed and the same tempo is applied throughout the rollout.

The readout and router use different representations from the same frozen VLM backbone. The readout performs a language-only forward pass once per instruction, whereas the router reuses visual features from the multimodal policy forward pass at every replan. Prototype construction uses text instructions, while router training uses existing task trajectories with phase-transition annotations. Neither component requires tempo-conditioned robot demonstrations or fine-tuning of the base VLA, and no external language model is used.

B. Prototype-Based Tempo Readout

1) *Contextual Features and Tempo Prototypes*: For each instruction, we perform a language-only forward pass

through the PaliGemma VLM backbone within the frozen $\pi_{0.5}$ policy. This pass uses the same VLM parameters as the multimodal policy pass but is executed only once per instruction. Valid language tokens attend bidirectionally to the instruction, allowing their representations to incorporate the surrounding linguistic context.

Motivated by prior evidence that behaviorally meaningful concepts can be recovered from internal transformer representations of pretrained VLAs [5], [17], we use the residual-stream output after the twelfth transformer block as a fixed contextual representation for tempo readout. Let $\mathbf{h}_i \in \mathbb{R}^{2048}$ denote the representation of valid language token i , excluding padding and boundary tokens, and let $\hat{\mathbf{h}}_i = \mathbf{h}_i / \|\mathbf{h}_i\|_2$ denote its normalized feature. We construct the readout using 30 task instructions from LIBERO-90 and reserve 10 additional instructions from distinct tasks for validation. For each training instruction, we prepend *quickly* to obtain a FAST example and *slowly* to obtain a SLOW example, while the unmodified instruction serves as a tempo-free example. This yields 30 contextual cue features for each tempo class and 30 tempo-free examples. Only the training examples are used to construct the prototypes and compute the readout thresholds.

Let $\mathcal{C}_F$ and $\mathcal{C}_S$ denote the sets of canonical cue-token indices for FAST and SLOW, respectively. For $c \in \{F, S\}$, we construct a unit-norm prototype [22]

$$\mathbf{p}_c = \text{normalize} \left(\frac{1}{|\mathcal{C}_c|} \sum_{i \in \mathcal{C}_c} \hat{\mathbf{h}}_i \right). \quad (1)$$

Thus, prototype construction requires only normalization and averaging and does not involve gradient-based readout training. For each token, we compute its cosine similarity to prototype c as $s_i^c = \hat{\mathbf{h}}_i^\top \mathbf{p}_c$ [23].

To distinguish tempo cues from ordinary instruction tokens, we calibrate one detection threshold per class. Let $\mathcal{N}$ contain all valid token features from the 30 tempo-free instructions. The threshold is defined as the midpoint between the mean similarity of the canonical cue features and that of the tempo-free tokens:

$$\tau_c = \frac{1}{2} \left(\frac{1}{|\mathcal{C}_c|} \sum_{i \in \mathcal{C}_c} s_i^c + \frac{1}{|\mathcal{N}|} \sum_{i \in \mathcal{N}} s_i^c \right). \quad (2)$$

The tempo-free term is computed over individual tokens rather than sentence-level averages. The prototypes and thresholds are fixed before evaluation.

2) *Token-Level Tempo Classification*: Each token is classified by combining its absolute similarity to the prototypes with competition between the FAST and SLOW classes:

$$y_i = \begin{cases} \text{FAST}, & s_i^F > \tau_F \wedge s_i^F > s_i^S, \\ \text{SLOW}, & s_i^S > \tau_S \wedge s_i^S > s_i^F, \\ \text{OTHER}, & \text{otherwise.} \end{cases} \quad (3)$$

Here, OTHER indicates that a token is not selected as a tempo cue and is distinct from NORMAL, which denotes the default execution tempo.

Because a word may be split into multiple subtokens, conflicting FAST and SLOW predictions can occur within the same word. We group subtokens using tokenizer surface and offset information. When a word contains both labels, we sum the threshold excess $s_i^c - \tau_c$ over the subtokens assigned to each class and assign the entire word to the class with the larger total excess. Ties are assigned OTHER. Words without a FAST-SLOW conflict retain their preliminary labels.

3) *Tempo Sequence Extraction*: After word-level correction, we identify contiguous regions of FAST and SLOW tokens. For each tempo class, we retain the region with the largest mean threshold excess. If both classes are present, the order of the selected region midpoints determines whether the decoded sequence is [FAST, SLOW] or [SLOW, FAST]. If only one class is present, the sequence contains that tempo alone; if neither class is detected, the readout returns [NORMAL].

The readout therefore decodes the requested order of tempo expressions rather than explicitly assigning each modifier to a particular verb or action. For multi-tempo instructions, the causal phase router determines which entry of the decoded sequence is active during execution.

C. Causal Phase Routing

When the readout returns a two-entry tempo sequence, TempoBridge must determine which task event is currently active. The causal phase router performs this alignment online without access to oracle task-phase information at deployment. It maintains recurrent state across replanning steps and permits only a single forward transition from Event 1 to Event 2. For single-entry sequences, the router is not used.

1) *Reused Visual Features and State History*: At replanning step r , we obtain a visual feature $\mathbf{v}_r \in \mathbb{R}^{2048}$ by mean-pooling the image-token outputs from the multimodal prefix pass already used for action generation. The router therefore does not require an additional visual-backbone forward pass. These visual features are distinct from the language-only features used by the tempo readout.

We project $\mathbf{v}_r$ to a 32-dimensional PCA representation $\bar{\mathbf{v}}_r$ and combine it with the robot state $\mathbf{q}_r \in \mathbb{R}^8$, which contains end-effector position, rotation, and gripper state. The router input is

$$\mathbf{x}_r = [\bar{\mathbf{v}}_r, \Delta \bar{\mathbf{v}}_r, \mathbf{q}_r, \Delta \mathbf{q}_r],$$

where $\Delta \bar{\mathbf{v}}_r = \bar{\mathbf{v}}_r - \bar{\mathbf{v}}_{r-1}$ and $\Delta \mathbf{q}_r = \mathbf{q}_r - \mathbf{q}_{r-1}$. Both differences are initialized to zero at $r = 0$. The PCA projection and component-wise standardization statistics are fitted on the router training data and fixed thereafter.

2) *Causal Transition Estimation*: A GRU processes the standardized router inputs and predicts a transition hazard h_r [24]. The hazards are accumulated into a monotonic transition probability [25]:

$$\mathbf{u}_r = \text{GRU}_\phi(\text{standardize}(\mathbf{x}_r), \mathbf{u}_{r-1}), \quad (4)$$

$$h_r = \sigma(\mathbf{w}^\top \mathbf{u}_r + b), \quad F_r = 1 - \prod_{j=0}^r (1 - h_j). \quad (5)$$

Execution begins in Event 1 and switches to Event 2 when $F_r \geq 0.5$. The transition is one-way, so Event 2 remains active for the remainder of the rollout.

3) *Interval-Supervised Router Training*: We train the router on 40 existing demonstrations from 20 LIBERO-90 tasks and validate it on 20 demonstrations from 10 additional tasks, using two demonstrations per task. The selected tasks cover object transfer, drawer manipulation, stove operation, and bowl stacking. Each demonstration is annotated with an interval containing the transition between the first and second task events. Transition intervals are first proposed automatically and then manually reviewed and corrected when necessary.

We use intervals rather than single transition timestamps because the semantic boundary between two manipulation events can be ambiguous. Let $\mathcal{D}_{\text{phase}}$ denote the annotated router-training set. For each training trajectory n , let $(L_n, R_n]$ denote the annotated interval containing the transition. Because F_r is the cumulative transition probability, the probability that the transition occurs within this interval is $F_{R_n} - F_{L_n}$ [26]. We therefore optimize

$$\mathcal{L}_{\text{phase}} = -\frac{1}{|\mathcal{D}_{\text{phase}}|} \sum_{n \in \mathcal{D}_{\text{phase}}} \log \left(F_{R_n}^{(n)} - F_{L_n}^{(n)} \right). \quad (6)$$

Only the router parameters are optimized, while the VLA feature extractor remains frozen. The router is trained exclusively on the 40 training demonstrations and remains fixed during deployment.

D. Tempo-Conditioned Action Modulation

At each replanning step, the tempo associated with the active event determines a modulation gain

$$\alpha(z) = \begin{cases} 1.3, & z = \text{FAST}, \\ 1.0, & z = \text{NORMAL}, \\ 0.7, & z = \text{SLOW}. \end{cases} \quad (7)$$

We use fixed heuristic gains symmetric around 1.0, without task-specific tuning, to test whether language-derived tempo cues induce consistent physical speed differences. For a single-entry tempo sequence, the same gain is used throughout execution. For a two-entry sequence, the gain changes when the phase router switches the active event.

For the LIBERO experiments, we instantiate tempo-conditioned modulation by scaling the Cartesian translation components of the nominal action chunk. Let $\Delta \mathbf{p}_{r,j} \in \mathbb{R}^3$ denote the translational component of action j at replanning step r , and let $\mathbf{b}_{r,j}$ contain the remaining rotation and gripper commands. TempoBridge applies

$$\tilde{\mathbf{a}}_{r,j} = [\alpha(z_{e_r})\Delta \mathbf{p}_{r,j}, \mathbf{b}_{r,j}], \quad j = 0, \dots, H-1, \quad (8)$$

where e_r denotes the active event. The gain is selected once per replan and applied uniformly to the predicted action chunk.

The robot executes the first $M \leq H$ actions of the modulated chunk, receives a new observation, and replans. Thus, in the LIBERO setting, TempoBridge changes the magnitude of the nominal translation commands while retaining the policy’s predicted rotation and gripper commands. The gain acts on commanded translation rather than directly prescribing physical TCP speed. The realized motion instead emerges from the closed-loop interaction between action modulation, execution, and subsequent replanning.

IV. EXPERIMENTS

We evaluate TempoBridge along four complementary axes. First, we test whether language-specified tempo can be realized in physical execution while preserving the underlying task competence. Second, we evaluate whether TempoBridge leaves the base policy largely unchanged when no tempo cue is present. Third, we test generalization to tempo expressions that are never used to construct or configure the readout. Finally, we analyze the individual contributions of language interpretation, phase routing, and motion realization, and validate the resulting tempo control on a physical robot.

A. Experimental Setup and Evaluation Metrics

We evaluate on 40 tasks from the four LIBERO [13] suites: Spatial, Object, Goal, and Long, with 10 tasks per suite. Both methods use the same $\pi_{0.5}$ [12] checkpoint trained for LIBERO. The baseline directly executes the nominal policy actions, whereas TempoBridge augments the same policy with the prototype-based tempo readout, causal phase router, and tempo-conditioned action modulation. The two

methods use identical instructions and matched initial states and random seeds.

The tempo prototypes are constructed only from task-context sentences containing *quickly* and *slowly*. The phase router is trained exclusively on existing demonstrations from LIBERO-90 tasks that are disjoint from all 40 evaluation tasks, using task-transition annotations but no tempo labels. No tempo-conditioned robot demonstrations are collected, and all $\pi_{0.5}$ parameters remain frozen. For LIBERO, TempoBridge scales the Cartesian translation components of the nominal action by gains of 1.3, 1.0, and 0.7 for FAST, NORMAL, and SLOW, respectively.

Task Success Rate. Task Success Rate measures the fraction of all rollouts satisfying the standard LIBERO task-success criterion, independently of tempo compliance.

Tempo Success Rate. Tempo Success Rate measures whether a successfully executed task realizes the requested relative tempo. We compute translational tool-center-point (TCP) speed from consecutive positions. For each evaluated segment, we discard the first and last 25% of speed samples in temporal order and average the central 50% to reduce boundary effects. For two-event instructions, evaluation boundaries are obtained from simulator-state predicates independently of the phase router. A rollout satisfies the tempo instruction when the mean TCP speed of the FAST event exceeds that of the SLOW event.

For single-event instructions, FAST and SLOW correspond to separate rollouts. We compare each task-successful rollout against task-successful rollouts with the opposite tempo under matched initial states and random seeds. When multiple opposite-tempo lexical variants are available, their mean successful speed is used as the reference. Tempo Success Rate is therefore computed over task-successful rollouts for which the required tempo comparison is available. It should be interpreted as a conditional measure of physical tempo realization after successful task execution.

Cue Recognition Accuracy. For unseen tempo expressions, we separately evaluate whether the readout assigns the expected tempo label. For two-event instructions, each position of the predicted tempo sequence is evaluated against its corresponding cue independently of the companion cue. For a single-event instruction, the readout must return the expected single tempo label. This metric separates readout errors from downstream execution errors.

B. Tempo Control under Canonical Instructions

We first evaluate explicit tempo control using the canonical *quickly* and *slowly* expressions. For 27 two-event tasks, we construct both FAST→SLOW and SLOW→FAST instructions. For the remaining 13 single-event tasks, we construct separate FAST and SLOW instructions. This yields 80 instructions, each evaluated over 10 rollouts, for a total of 800 rollouts per method.

Table I shows that TempoBridge increases the overall Tempo Success Rate from 52.6% to 89.7%, while the Task Success Rate remains comparable to the baseline (92.9% vs. 93.6%). Because Tempo Success Rate is evaluated only

TABLE I
TASK, TEMPO, AND TASK & TEMPO SUCCESS RATES (%) UNDER *quickly/slowly* INSTRUCTIONS.

Method	Spatial		Object		Goal		Long		Overall	
	Task Success Rate $\uparrow$	Tempo Success Rate $\uparrow$	Task Success Rate $\uparrow$	Tempo Success Rate $\uparrow$	Task Success Rate $\uparrow$	Tempo Success Rate $\uparrow$	Task Success Rate $\uparrow$	Tempo Success Rate $\uparrow$	Task Success Rate $\uparrow$	Tempo Success Rate $\uparrow$
$\pi_{0.5}$	94.0	48.9	96.0	49.5	93.0	57.8	91.5	54.4	93.6	52.6
TempoBridge	94.5	91.0	97.5	90.8	92.5	93.8	87.0	82.9	92.9	89.7

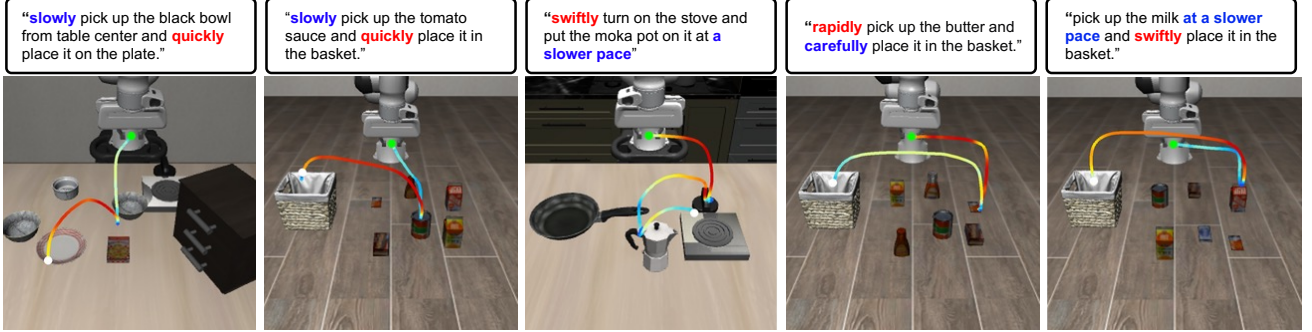


Fig. 3. Qualitative examples of TempoBridge execution on LIBERO. The left two panels use *quickly/slowly* cues, while the right three use tempo expressions unseen during readout construction. Each panel shows the instruction and executed TCP trajectory, colored by measured TCP speed from slower motion (blue) to faster motion (red).

TABLE II
TASK SUCCESS RATES (%) ON LIBERO BENCHMARK INSTRUCTIONS WITHOUT TEMPO CUES.

Method	Spatial	Object	Goal	Long	Overall
$\pi_{0.5}$	95.0	97.0	95.0	93.0	95.0
TempoBridge	98.0	96.0	95.0	88.0	94.3

among task-successful, tempo-evaluable rollouts, we additionally report Task & Tempo Success Rate, defined over all rollouts. Under this metric, TempoBridge improves from 48.8% to 82.0%, showing that the gain in tempo compliance is retained when task completion and tempo compliance are evaluated jointly.

C. Task Competence without Tempo Cues

We next evaluate whether TempoBridge preserves the underlying policy behavior when the instruction contains no explicit tempo cue. We use the 40 original LIBERO instructions without added tempo expressions and evaluate 10 rollouts per instruction. The tempo readout returns NORMAL for 38 of the 40 instructions (95%), leaving the nominal translation commands unchanged for those instructions. As shown in Table II, TempoBridge achieves an overall Task Success Rate of 94.3%, compared with 95.0% for the baseline. These results show that TempoBridge largely preserves the task competence of the underlying policy when no tempo modification is requested.

D. Generalization to Unseen Tempo Expressions

Because TempoBridge reads tempo cues from contextual representations of the frozen VLM rather than relying on a fixed lexical rule set, we test whether the readout can transfer to tempo expressions that are not used during its construction or configuration. We use *rapidly* and *swiftly* as

FAST cues, and *carefully* and *at a slower pace* as SLOW cues. All TempoBridge components remain fixed throughout this evaluation. For each two-event task, we combine each FAST expression with each SLOW expression and assign them to the two task clauses in both FAST $\rightarrow$ SLOW and SLOW $\rightarrow$ FAST orders, yielding eight instructions per task. For each single-event task, we prepend each of the four unseen expressions to the benchmark instruction. This produces 216 two-event instructions evaluated over 10 rollouts each and 52 single-event instructions evaluated over 20 rollouts each, for a total of 3,200 rollouts per method.

As shown in Table III, TempoBridge increases Tempo Success Rate from 53.5% to 80.1% while maintaining a Task Success Rate of 92.3%. The Task & Tempo Success Rate also improves from 50.6% to 72.9%. Although tempo performance is lower than in the canonical setting, these results show that TempoBridge retains substantial tempo control for expressions that were never used to construct the readout.

To better understand this degradation, Table IV reports cue-level recognition accuracy. The readout recognizes *rapidly* and *swiftly* as FAST with 98.3% accuracy each, and *at a slower pace* as SLOW with 95.0% accuracy, whereas accuracy for *carefully* drops to 57.9%. The high recognition rates for expressions with explicit tempo semantics indicate that the frozen VLM representations support transfer beyond the canonical cues used for readout construction, while the lower accuracy for *carefully* suggests that generalization is more challenging when the relation to execution speed is semantically indirect. Representative TempoBridge executions with unseen expressions are shown in Fig. 3, while additional qualitative comparisons with the $\pi_{0.5}$ baseline are provided on the project page.

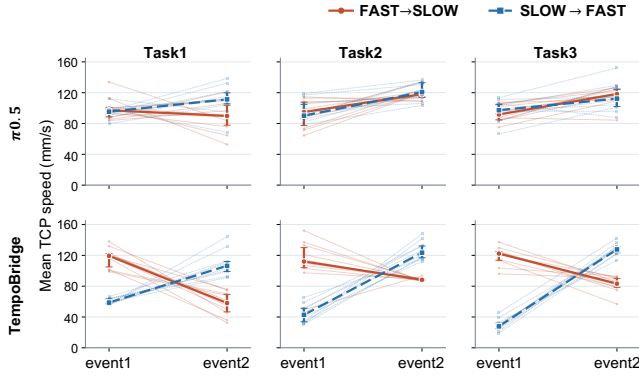


Fig. 5. Real-world TCP speed under FAST→SLOW (FS) and SLOW→FAST (SF) instructions. Columns correspond to the three tasks, with the task-adapted $\pi_{0.5}$ baseline in the top row and TempoBridge in the bottom row. Thin lines represent individual task-successful rollouts, connecting the mean TCP speeds of Event 1 and Event 2. Bold lines indicate the median, and error bars show the interquartile range over 10 successful rollouts per condition.

V. CONCLUSIONS

We presented TempoBridge, a lightweight framework for making language-specified execution tempo actionable in a frozen VLA. Rather than learning a tempo-conditioned policy from additional speed-conditioned robot data, TempoBridge leverages linguistic structure available in the pre-trained VLM representation and connects it to phase-aware motion modulation. Across 40 LIBERO tasks, the method substantially improves tempo compliance while largely preserving task competence, maintains near-baseline performance when no tempo cue is present, and generalizes to previously unseen tempo expressions. Real-world experiments further demonstrate that language-specified tempo can be used to drive measurable changes in execution speed on a physical robot. These results suggest that pretrained VLA representations can provide a useful interface not only for specifying *what* a robot should do, but also for controlling aspects of *how* the behavior is executed.

TempoBridge currently focuses on discrete FAST/SLOW tempo control and relatively simple task-phase structures. Future work could extend the framework to continuous tempo control, more complex multi-stage tasks, and broader execution attributes beyond speed. More generally, our results motivate further study of how semantic information available in pretrained VLA representations can be used as an interface to explicit execution-time control mechanisms without modifying the underlying policy.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *CoRL*, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong *et al.*, “Openvla: An open-source vision-language-action model,” in *CoRL*, 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *RSS*, 2025.
- [4] D. Jing, J. Nie, T. Zhang, J. Liu, H. Yao, Z. Lu, and M. Ding, “Tempovla: Learning speed-controllable vision-language-action policies,” *arXiv preprint arXiv:2606.06491*, 2026.

- [5] B. Häon, K. C. Stocking, I. Chuang, and C. Tomlin, “Mechanistic interpretability for steering vision-language-action models,” in *CoRL*, 2025.
- [6] Y. Shao, X. Huang, B. Zhang, Q. Liao, Y. Gao, Y. Chi, Z. Li, S. Shao, and K. Sreenath, “Langwbc: Language-directed humanoid whole-body control via end-to-end learning,” in *RSS*, 2025.
- [7] S. Belkhal, T. Ding, T. Xiao, P. Sermanet, Q. Vuong, J. Tompson, Y. Chebotar, D. Dwibedi, and D. Sadigh, “Rt-h: Action hierarchies using language,” in *RSS*, 2024.
- [8] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis *et al.*, “Droid: A large-scale in-the-wild robot manipulation dataset,” *arXiv preprint arXiv:2403.12945*, 2024.
- [9] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0,” in *ICRA*. IEEE, 2024.
- [10] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch, A. W. He, V. Myers, M. J. Kim, M. Du *et al.*, “Bridgedata v2: A dataset for robot learning at scale,” in *CoRL*. PMLR, 2023, pp. 1723–1736.
- [11] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [12] Physical Intelligence, K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *CoRL*, 2025.
- [13] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.
- [14] D. Ghosh, H. R. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejma, T. Kreiman, C. Xu, J. Luo *et al.*, “Octo: An open-source generalist robot policy,” in *RSS*, 2024.
- [15] C. Glossop, W. Chen, A. Bhorkar, D. Shah, and S. Levine, “Cast: Counterfactual labels improve instruction following in vision-language-action models,” *RA-L*, vol. 11, no. 10, pp. 11 785–11 792, 2026.
- [16] L. Wenjie, Y. Jangir, I. Stepka, Y. Agarwal, M. Kipsang, and Y. Bisk, “Lost in reconstruction: Aligning action representations with language in vision-language-action models,” *arXiv preprint arXiv:2608.10484*, 2026.
- [17] H. Buurmeijer, C. A. Alonso, A. Swann, and M. Pavone, “Observing and controlling features in vision-language-action models,” in *Mechanistic Interpretability Workshop at ICML 2026*, 2026.
- [18] M. A. Khan, N. Boskov, F. M. Anwar, and M. A. Khan, “Controlling vision-language-action policies through sparse latent directions,” in *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025.
- [19] A. Babu, R. Kaur, N. D. Bastian, O. Kotevska, S. Jha, Y. Wu, S. K. Jha, and A. Roy, “CTRL-STEER: Closed-loop neuron activation control in vision-language-action models,” in *Third Workshop on Visual Concepts at CVPR 2026*, 2026.
- [20] P. Khayatan, S. Meziane, J. Parekh, and M. Cord, “Dimas: Distribution matching for steering vision-language-action models,” in *EXCV Workshop at ECCV 2026*, 2026.
- [21] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello *et al.*, “Paligemma: A versatile 3b vlm for transfer,” *arXiv preprint arXiv:2407.07726*, 2024.
- [22] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” *Advances in neural information processing systems*, vol. 30, 2017.
- [23] O. Vinyals, C. Blundell, T. Lillicrap, D. Wierstra *et al.*, “Matching networks for one shot learning,” *NeurIPS*, vol. 29, 2016.
- [24] K. Cho, B. Van Merriënboer, Ç. Gulçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” in *EMNLP*, 2014.
- [25] H. Kvamme and Ø. Borgan, “Continuous and discrete-time survival prediction with neural networks,” *Lifetime data analysis*, vol. 27, no. 4, pp. 710–736, 2021.
- [26] B. W. Turnbull, “The empirical distribution function with arbitrarily grouped, censored and truncated data,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 38, no. 3, pp. 290–295, 1976.